



**Hochschule
Bonn-Rhein-Sieg**

*University
of Applied Sciences*

Fachbereich Informatik
Department of Computer Sciences

Abschlussarbeit

Studiengang Bachelor Informatik

Entwurf und Evaluation eines fehlertoleranten Job-Processing in Java

von

Max Urfei

Erstprüfer: Prof. Dr. Andreas Hackelöer

Zweitprüfer: M. Sc. Moritz Balg

eingereicht am TT.MM.YYYY

Inhaltsverzeichnis

Abkürzungsverzeichnis	iii
Abbildungsverzeichnis	iv
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	1
1.3 Zielsetzung	2
1.4 Hypothesen	3
1.5 Aufbau der Arbeit	3
2 Grundlagen	4
2.1 Grundlagen der Hintergrundverarbeitung	4
2.1.1 Asynchrone Verarbeitung und Queueing	4
2.1.2 Ausführungssemantiken	6
2.1.3 Background Job-Processing	7
2.2 Konsistenz und konkurrierende Verarbeitung	8
2.2.1 ACID	9
2.2.2 Transaktionsgrenzen	10
2.2.3 Isolationsstufen und Synchronisationsverfahren	11
2.3 Fehlertoleranz und Wiederherstellung	12
2.3.1 Fehlerklassifikation	12
2.3.2 Retry-/Backoff-Strategien	13
2.3.3 Recovery	15
2.3.4 Idempotenz	16
2.4 Stand der Forschung	17
2.4.1 Brokerbasierte Ansätze	17
2.4.2 Datenbankbasierte Ansätze	18
2.4.3 Einordnung und Forschungslücke	18
3 Methodik	20
3.1 Literaturrecherche	20
3.2 Ergebniserzeugung	20
4 Anforderungen	21
4.1 Funktionale Anforderungen	21
4.2 Qualitätsanforderungen	21
4.3 Zusicherungen	21
5 Umsetzung	22
5.1 Architektur und Design	22
5.1.1 Zustandsmodell eines Jobs	24

5.2	Implementierung	24
5.2.1	Technologie-Stack	24
5.2.2	Zentrale Code-Entscheidungen	24
6	Evaluation	25
6.1	Fehlerszenarien	25
6.2	Durchführung	25
6.3	Ergebnisse	25
7	Diskussion	26
7.1	Gewonnene Erkenntnisse	26
7.2	Einschränkungen und Grenzen der Arbeit	26
8	Fazit und Ausblick	27
	Literaturverzeichnis	28

Abkürzungsverzeichnis

ACID Atomicity, Consistency, Isolation, Durability. 9–11, 18

CRUD Create, Read, Update, Delete. 1

REST Representational State Transfer. 7

UI User Interface. 7, 8

UUID Universally Unique Identifier. 17

Abbildungsverzeichnis

2.1	Vergleich synchroner und asynchroner Kommunikation	5
2.2	Queueing	5
2.3	Verhalten bei transienten Fehlern	14

1 Einleitung

1.1 Motivation

In modernen Softwaresystemen stellt die Verarbeitung von Hintergrundjobs einen zentralen Bestandteil vieler Backend-Architekturen dar (Kleppmann 2017, Kap. 10). Zahlreiche Geschäftsprozesse, darunter die Generierung von Reports, der Import großer Datenmengen, periodische Abrechnungsläufe oder das Versenden asynchroner Benachrichtigungen, erfordern eine Verarbeitung, die entkoppelt vom synchronen Request-Response-Zyklus stattfindet. Insbesondere in industrienahen und sicherheitskritischen Anwendungen, etwa im Bereich der luftfahrttechnischen Wartungsdokumentation, müssen solche Prozesse nicht nur funktional korrekt ablaufen, sondern darüber hinaus zuverlässig, nachvollziehbar und fehlertolerant sein. Aber auch in nicht-kritischen Systemen ist Zuverlässigkeit von Bedeutung für die Nutzerfreundlichkeit der Anwendung, als auch um Produktivitäts- und Gewinneinbußen in Produktivsystemen in Folge von Fehlern zu vermeiden (Kleppmann 2017, Kap. 1 Abschnitt „How Important Is Reliability?“).

1.2 Problemstellung

In der Praxis können bei der Verarbeitung von Hintergrundjobs jederzeit Teilausfälle auftreten. Prozesse können während der Verarbeitung unerwartet terminieren, Datenbankverbindungen können kurzzeitig unterbrochen werden oder identische Anfragen werden zum Beispiel infolge von clientseitigen Wiederholungsversuchen mehrfach an das System übermittelt oder Ergebnisse bzw. Antworten gehen verloren (Kleppmann 2017, Kap. 8). Ohne geeignete Mechanismen führen derartige Szenarien zu inkonsistenten Systemzuständen. Beispielsweise verbleiben Jobs im Status „laufend“, obwohl kein Worker sie mehr aktiv verarbeitet, Ergebnisse werden doppelt erzeugt oder Aufträge gehen vollständig verloren.

Bestehende Frameworks und Bibliotheken, etwa Quartz Scheduler (Terracotta Inc. (Hrsg.) 2026) oder Spring Batch (Broadcom Inc. (Hrsg.) 2026b), stellen umfangreiche Funktionalitäten für die Hintergrundverarbeitung bereit. Hierzu zählen unter anderem Scheduling, Wiederholungsmechanismen sowie Verfahren zur Fehlerbehandlung und Wiederaufnahme. Sie verfolgen dabei jedoch jeweils eigene Architekturansätze und abstrahieren die zugrunde liegenden Mechanismen weitgehend hinter ihren Programmierschnittstellen. Dadurch eignen sie sich nur eingeschränkt, um systematisch zu untersuchen, welche Architekturentscheidungen für bestimmte Konsistenz- und Verarbeitungszusicherungen tatsächlich erforderlich sind.

Die beschriebene Problematik ist von hoher praktischer und industrieller Relevanz. Nahezu jedes Backend-System, das über einfache Create, Read, Update, Delete (CRUD)-Operationen hinausgeht, muss Aufgaben zuverlässig im Hintergrund verarbeiten und dabei konsistente Datenbestände gewährleisten (Richardson 2018, Kap. 4). Fehler in diesem Bereich führen zu Datenverlust, inkonsistenten Geschäftszuständen sowie unnöti-

gem manuellem Eingriff und können insbesondere in regulierten Domänen wie der Luftfahrt Compliance-Verstöße nach sich ziehen. Auch im Kontext von Cloud-nativen Architekturen ist dieses Thema relevant. Horizontale Skalierung, Container-Neustarts und kurzzeitige Netzwerkunterbrechungen gehören dort zum regulären Betriebsverhalten und müssen bereits beim Entwurf eines Systems berücksichtigt werden. Daher bilden die in dieser Arbeit behandelten Konzepte auch die Grundlage für den zuverlässigen Betrieb von Backend-Systemen in Cloud-Umgebungen.

Obwohl die zugrundeliegenden Konzepte wie Fehlertoleranz, Recovery, Idempotenz, Nebenläufigkeitskontrolle und Transaktionsmanagement in der Literatur umfassend beschrieben werden, existieren nur wenige Arbeiten, die diese Aspekte zu einer durchgängigen datenbankbasierten Architektur für Background-Job-Processing zusammenführen und deren Verhalten unter definierten Fehlerszenarien systematisch evaluieren. Ebenso implementieren bestehende Frameworks diese Mechanismen häufig als interne Abstraktionen, wodurch deren Zusammenspiel und Auswirkungen auf die Zuverlässigkeit des Gesamtsystems für Entwickler nur eingeschränkt nachvollziehbar sind.

Daraus ergibt sich die dieser Arbeit zugrundeliegende Hauptforschungsfrage:
Welche Architekturmechanismen und Entwurfsentscheidungen sind notwendig, um in einem Java/Spring-basierten Job-Processing-Backend definierte Konsistenzzusicherungen unter typischen Teilausfallszenarien nachweisbar einzuhalten?

Zur Beantwortung dieser Frage werden folgende Teilfragen untersucht:

- TF1: Welche Zusicherungen muss das System gewährleisten, damit Jobs stets korrekt und ohne Inkonsistenzen verarbeitet werden?
- TF2: Welchen Beitrag leisten Idempotenz, Retry-Strategien und Recovery-Mechanismen zur Robustheit unter definierten Fehlerszenarien?
- TF3: Wie müssen Transaktionen und Locking-Strategien genutzt werden, damit bei paralleler Arbeit mehrerer Worker keine Doppelverarbeitung oder inkonsistenten Zustände entstehen?
- TF4: Wie lässt sich sicherstellen, dass Jobs nach einem unerwarteten Prozessabbruch korrekt wiederaufgenommen werden, ohne dass Aufträge verloren gehen?

1.3 Zielsetzung

Ziel dieser Arbeit ist der Entwurf, die prototypische Implementierung und die Evaluation eines fehlertoleranten Job-Processing-Backends auf Basis einer relationalen Datenbank in Kombination mit Java und Spring. Dabei soll herausgearbeitet werden, welche Architekturmechanismen erforderlich sind, damit Hintergrundjobs auch bei Teilausfällen zuverlässig, korrekt und nachvollziehbar verarbeitet werden und definierte Konsistenzzusicherungen eingehalten werden.

Konkret verfolgt die Arbeit folgende Teilziele:

- TZ1: Definition von Zusicherungen: Es sollen präzise Korrektheitsbedingungen formuliert werden, die das System zu jedem Zeitpunkt einhalten muss, unabhängig davon, welche Fehler auftreten. Dazu zählen unter anderem, dass kein Job doppelt erfolgreich abgeschlossen wird, dass kein Job dauerhaft verloren geht und dass Statusübergänge ausschließlich dem definierten Zustandsmodell folgen.
- TZ2: Architekturentwurf: Es soll eine Architektur entworfen werden, die persistente Job-States, Idempotenz, Retry-Strategien sowie ein nachvollziehbares Statusmodell umfasst. Architekturentscheidungen sollen explizit dokumentiert und begründet werden.
- TZ3: Prototypische Implementierung: Die entworfene Architektur soll als funktionsfähiger Prototyp umgesetzt werden, der die definierten Zusicherungen unter realistischen Bedingungen einhält.
- TZ4: Evaluation unter Fehlerbedingungen: Es soll nachgewiesen werden, dass das System seine Zusicherungen auch bei gezielt herbeigeführten Teilausfällen nicht verletzt.

1.4 Hypothesen

Auf Basis der Forschungsfrage und der Zielsetzung werden folgende Hypothesen formuliert, deren Überprüfung im Rahmen der Implementierung und insbesondere der Evaluation erfolgt.

- H1: Durch die Kombination eines persistenten Statusmodells mit transaktionsbasiertem Locking und Idempotenz-Mechanismen lässt sich ein System entwerfen, das auch bei unerwarteten Prozessabbrüchen keine Jobs verliert und keine doppelten Ergebnisse erzeugt.
- H2: Retry-Strategien mit exponentiellem Backoff in Verbindung mit einer Fehlerklassifikation (transient vs. permanent) erhöhen die Robustheit des Systems gegenüber kurzzeitigen Infrastrukturausfällen, ohne dabei die definierten Zusicherungen zu verletzen.
- H3: Die definierten Zusicherungen lassen sich durch automatisierte Tests mit gezielter Fehlerinjektion reproduzierbar überprüfen und nachweisen.

1.5 Aufbau der Arbeit

2 Grundlagen

Dieses Kapitel vermittelt die theoretischen Grundlagen, die für das Verständnis der im weiteren Verlauf entwickelten Architektur erforderlich sind. Zunächst werden die grundlegenden Konzepte der Hintergrundverarbeitung sowie die Anforderungen an zuverlässige Background-Job-Processing-Systeme erläutert. Darauf aufbauend werden die für diese Arbeit relevanten Mechanismen zur Gewährleistung von Konsistenz bei paralleler Verarbeitung sowie Konzepte zur Fehlertoleranz, Wiederherstellung und idempotenten Verarbeitung eingeführt. Die dargestellten Grundlagen bilden die Basis für den in Kapitel 5.1 vorgestellten Architekturentwurf und dessen anschließende Implementierung und Evaluation.

2.1 Grundlagen der Hintergrundverarbeitung

Hintergrundverarbeitung dient dazu, zeitintensive oder ressourcenintensive Aufgaben vom synchronen Request-Response-Zyklus zu entkoppeln. Gleichzeitig entstehen daraus jedoch auch einige Herausforderungen: In verteilten Umgebungen sind Teilausfälle, Timeouts und kurzzeitige Unterbrechungen üblich, sodass ohne entsprechende Mechanismen nicht davon ausgegangen werden kann, dass eine angestoßene Verarbeitung vollständig und genau einmal ausgeführt wird. Insbesondere kann bei Kommunikationsabbrüchen oder Wiederholungsversuchen unklar bleiben, ob eine Operation bereits verarbeitet wurde oder ob sie erneut angestoßen werden muss (Kleppmann 2017, Kap. 8). Vor diesem Hintergrund werden in den folgenden Abschnitten grundlegende Konzepte vorgestellt, die eine Entkopplung der Verarbeitung ermöglichen und die Basis für robuste Hintergrundverarbeitung bilden.

2.1.1 Asynchrone Verarbeitung und Queueing

In Backend-Systemen lässt sich Kommunikation und Verarbeitung bezüglich der zeitlichen Kopplung zwischen synchronem und asynchronem Verhalten unterscheiden. Der Unterschied der Funktionsweise wird in Abbildung 2.1 deutlich. Bei synchroner Verarbeitung nach dem Request-Response-Stil wartet der Aufrufer auf eine Antwort auf die Anfrage und blockiert gegebenenfalls bevor er fortfährt. Bei asynchroner Verarbeitung wird die Antwort auf den Auftrag vom Zeitpunkt des Erhalts entkoppelt. Der Aufrufer initiiert die Arbeit, blockiert jedoch nicht und kann auch während des Wartens auf eine Antwort weiter arbeiten (Richardson 2018, Kap. 3.1.1).

Zur praktischen Umsetzung asynchroner Verarbeitung werden häufig Puffer eingesetzt. Dieses Prinzip wird als Queueing bezeichnet. Dadurch können Nachrichtenverluste reduziert und Lastspitzen ausgeglichen werden, indem der Puffer erhaltene Anfragen zur späteren Ausführung zwischenspeichert (Kleppmann 2017, Kap. 11, Abschnitt „Message brokers“). Das grundlegende Prinzip ist in Abbildung 2.2 schemenhaft dargestellt. Ein solcher Puffer kann verschiedene Ausprägungen annehmen, beispielsweise als In-Memory beim Empfänger, als Datenbank oder in Form eines Message Brokers. Ein Produzent

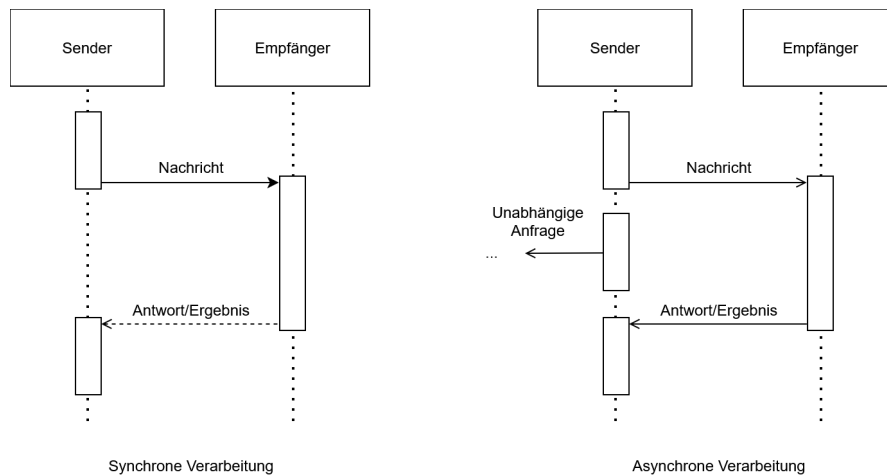


Abbildung 2.1: Vergleich synchroner und asynchroner Kommunikation

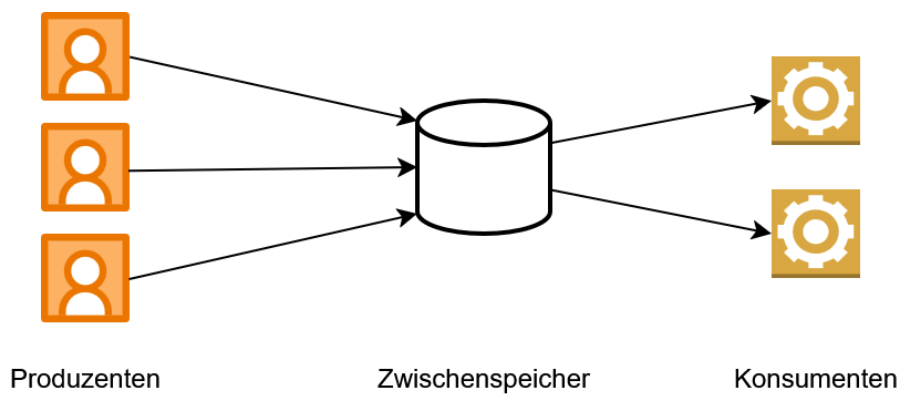


Abbildung 2.2: Queueing

sendet eine Nachricht an einen Zwischenspeicher und wartet in der Regel lediglich auf das Acknowledgment, dass die Nachricht zwischengespeichert wurde, nicht jedoch auf die Verarbeitung oder das Ergebnis (Kleppmann 2017, Kap. 11, Abschnitt „Messaging Systems“). Die so erreichte Entkopplung hat gleichzeitig den Vorteil, dass die Verarbeitung der Anfrage unabhängig von der Verbindung zwischen Produzent und Konsument wird. Im Gegensatz dazu kann bei synchronen Systemen bei Verbindungsabbrüchen oder Timeouts unklar bleiben, ob eine Anfrage den Server erreicht hat oder bereits verarbeitet wurde. Genau dieses Problem adressieren Message Broker: Falls ein Konsument temporär nicht verfügbar ist oder während der Verarbeitung ausfällt, können Message Broker trotzdem für die At-Least-Once-Semantik (vgl. Kapitel 2.1.2) sorgen (Kleppmann 2017, Kap. 11, Abschnitt „Acknowledgments and redelivery“). Des Weiteren müssen Verarbeitungskapazitäten in Folge von Queueing nicht auf die maximal zu erwartende Last ausgelegt werden, sondern können sich an der durchschnittlichen Auslastung orientieren.

Kurzzeitige Lastspitzen werden dabei durch die Zwischenspeicherung der Aufgaben in der Queue ausgeglichen (Microsoft Corporation (Hrsg.) 2026b). Die synchrone Request-Response-Kommunikation bietet demgegenüber den Vorteil einer unmittelbaren Rückmeldung über Erfolg oder Fehler einer Operation, setzt jedoch voraus, dass Client und Server während der gesamten Interaktion verfügbar sind (Richardson 2018, Kap. 3.4).

Die asynchrone Verarbeitung bildet damit die Grundlage für die Hintergrundverarbeitung, indem die Entkopplung genutzt wird, um Aufgaben unabhängig vom ursprünglichen Aufruf auszuführen und gleichzeitig die Robustheit und Skalierbarkeit des Systems zu erhöhen (Newman 2026, Kap. 8, Abschnitt „Why Use A Message Broker?“).

2.1.2 Ausführungssemantiken

In verteilten Systemen führen potenziellen Prozessabstürze und Teilausfälle dazu, dass Aufträge verloren gehen oder der Verarbeitungszustand einer Aufgabe unklar sein kann. Um zu beschreiben, was Systeme aufgrund ihres Entwurfs und ihrer Mechanismen zur Fehlertoleranz bezüglich Auslieferung beziehungsweise Ausführung garantieren, werden grundsätzlich drei fundamentale Ausführungssemantiken unterschieden, die Newman (2026, Kap. 8 Abschnitt „Types of Delivery Guarantees“) vor dem Hintergrund der Auslieferung wie folgt definiert. Im Kontext von Job-Processing sind diese Semantiken zur Nachrichtenzustellung analog auf die Ausführung der Jobs anwendbar.

At-Least-Once

Ein Konsument erhält die Nachricht mindestens einmal. Bei der Auslieferung bestätigt der Konsument über ein Acknowledge das Ankommen der Nachricht. Sofern das Acknowledgement beim Absender eingeht, ist für diesen garantiert, dass die Nachricht angekommen ist. Bleibt diese Bestätigung innerhalb eines definierten Zeitfensters jedoch aus, z.B. weil die Nachricht oder das Acknowledgement verloren geht, wird die Nachricht erneut zugestellt. Dieser Fall wird Redelivery genannt. Dieses Verfahren verhindert Datenverlust, birgt jedoch das Risiko von Duplikaten, für den Fall, dass die Nachricht ankommt, aber das Acknowledge verloren geht und es infolgedessen zur Redelivery kommt. Einschränkung sei hier gesagt, dass die Zustellung nicht garantiert werden kann, wenn der Konsument bei keinem Zustellversuch verfügbar ist.

At-Most-Once

Ein Konsument erhält die Nachricht maximal einmal. Der Produzent schickt die Nachricht ab und erwartet keine Empfangsbestätigung. Eine erneute Zustellung findet unter keinen Umständen statt. Dadurch werden Duplikate strikt ausgeschlossen, jedoch muss ein potenzieller Datenverlust zugunsten besserer Latenzen hingenommen werden.

Exactly-Once

Diese Semantik garantiert, dass eine Nachricht genau einmal beim Konsumenten ankommt. In der Praxis ist das nur schwer zu realisieren, da beispielsweise nicht klar sein

kann, ob ein Acknowledgement nur noch nicht angekommen, oder schon verloren gegangen ist. Viele Umsetzungen beruhen daher auf der Notwendigkeit, dass sowohl beim Produzent, als auch Konsument entsprechende Mechanismen implementiert werden. Eine Variante zur Umsetzung von Exactly-Once ist die Approximation über eine Kombination der Implementierung von At-Least-Once beim Produzent und Idempotenz beim Konsument. Die Garantie bezieht sich somit nicht auf die physische Zustellung einer Nachricht, sondern auf die Wirkung ihrer Verarbeitung. Eine Nachricht kann mehrfach zugestellt werden, solange ihre Verarbeitung nur einmal wirksam wird.

2.1.3 Background Job-Processing

Aufbauend auf den Konzepten der asynchronen Verarbeitung (vgl. Kapitel 2.1.1) bezeichnet das Background Job-Processing die Ausführung von Aufgaben außerhalb des Request-Response-Zyklus. Hierbei werden fachliche Operationen als eigenständige Arbeitseinheiten (Jobs) modelliert und zeitlich entkoppelt verarbeitet.

In der Softwarearchitektur existieren verschiedene etablierte Entwurfsmuster zur Umsetzung einer solchen Infrastruktur, wie beispielsweise das Queue-Based Load Leveling Pattern (Microsoft Corporation (Hrsg.) 2026b). Dieses Muster besteht ähnlich des Konzepts aus Abbildung 2.2 aus drei funktionalen Kernkomponenten:

- **Der Produzent:** Erzeugt einen Job und gibt diesen weiter an den Job-Store. Das kann zum einen direkt durch das User Interface (UI) geschehen, oder die UI ruft einen Endpoint auf, der den Job über eine synchrone Schnittstelle, wie z.B. Representational State Transfer (REST), entgegen nimmt. In letzterem Fall fungiert der Service hinter dem Endpoint als Produzent. Dessen Aufgabe besteht darin, den Auftrag zu validieren und an die Persistenzschicht zu übergeben, woraufhin der synchrone Request-Response-Zyklus für den Client sofort beendet wird. Jobs können aber nicht nur durch solche Event-driven Trigger ausgelöst werden, sondern auch durch Schedule-driven Trigger. Dabei erfolgt die Ausführung eines Jobs periodisch oder zu vordefinierten Zeitpunkten, z.B. über durch einen Cron-Trigger (Microsoft Corporation (Hrsg.) 2026a).
- **Der Job-Store:** Speichert die Jobs für den Zeitraum von der Erstellung bis zur endgültigen Beendigung zwischen. Für die Umsetzung existieren verschiedene Varianten, beispielsweise als Message Broker oder relationale Datenbank. Diese Realisierungsmöglichkeiten bieten jeweils spezifische Vor- und Nachteile in der Nutzung. Die Auswahl des passenden Ansatzes sollte daher durch die Qualitätsanforderungen an das System, wie z.B. Durchsatz oder Akzeptanz einzelner verlorener Jobs, begründet werden (Kleppmann 2017, Kap. 11).
- **Der Worker:** Autonome Komponente, die als Konsument fungiert. Sobald der Worker über freie Systemressourcen verfügt, kann er einen wartenden Job aus dem Job-Store beanspruchen und führt diesen aus (Microsoft Corporation (Hrsg.) 2026b).

Durch eine solche Architektur können Background Jobs als „fire and forget“-Operationen (Microsoft Corporation (Hrsg.) 2026a) implementiert werden. Sie laufen asynchron in einem vom Client isolierten Prozess, sodass deren Verarbeitungszeit keinerlei unmittelbare Auswirkung auf die Reaktionszeit oder Verfügbarkeit der UI hat (Microsoft Corporation (Hrsg.) 2026a). Außerdem ermöglicht dieses Konzept das Abfangen von Lastspitzen und flexiblere Skalierung (Microsoft Corporation (Hrsg.) 2026b).

Die Zuweisung von Jobs an Worker kann grundsätzlich über unterschiedliche Modelle erfolgen. Beim Push-Modell weist eine zentrale Komponente Aufgaben aktiv freien Worker-Instanzen zu. Beim Pull-Modell fragen Worker eigenständig neue Aufgaben aus dem Job-Store ab. Beide Ansätze unterscheiden sich hinsichtlich Komplexität, Skalierbarkeit und Koordinationsaufwand (Kleppmann 2017, Kap. 11 Abschnitt „Transmitting Event Streams“).

Werden mehrere Worker parallel betrieben, entsteht die Herausforderung der konkurrierenden Verarbeitung. Ohne geeignete Koordinationsmechanismen könnten mehrere Instanzen denselben Job gleichzeitig beanspruchen oder widersprüchliche Statusänderungen durchführen. Die hierfür relevanten Konzepte werden in Kapitel 2.2 behandelt.

Da die Verarbeitung zeitlich entkoppelt erfolgt, müssen Informationen über offene und bereits bearbeitete Jobs über den Lebenszyklus einzelner Prozesse hinweg erhalten bleiben. Hierfür ist eine persistente Speicherung des Verarbeitungszustands erforderlich. Insbesondere in verteilten Systemen können Teilausfälle auftreten, bei denen einzelne Komponenten oder Prozesse während der Ausführung ausfallen, während andere Teile des Systems weiterhin verfügbar bleiben. Damit eine unterbrochene Verarbeitung nach einem solchen Ausfall wieder aufgenommen werden kann, muss das System den zuletzt bekannten Zustand kennen oder anhand persistenter Informationen rekonstruieren können (Kleppmann 2017, Kap. 11, Abschnitt „Rebuilding state after a failure“).

Neben Prozessabstürzen können auch externe Dienste vorübergehend nicht erreichbar sein oder Kommunikationsfehler auftreten. Das System muss daher abhängig von der jeweiligen Fehlersituation entscheiden können, ob eine Verarbeitung erneut durchgeführt, fortgesetzt oder endgültig beendet werden soll (Microsoft Corporation (Hrsg.) 2026a). Die hierfür relevanten Konzepte zur Fehlerbehandlung und Wiederherstellung werden in Kapitel 2.3 vorgestellt.

Da sowohl Wiederholungsversuche als auch Wiederaufnahmen nach einem Ausfall dazu führen können, dass einzelne Verarbeitungsschritte mehrfach ausgeführt werden, muss darüber hinaus sichergestellt werden, dass solche Mehrfachausführungen keine inkonsistenten Ergebnisse verursachen. Hierzu werden Verfahren der idempotenten Verarbeitung eingesetzt, die in Kapitel 2.3.4 vorgestellt werden.

2.2 Konsistenz und konkurrierende Verarbeitung

Bei der Verarbeitung von Daten durch mehrere parallel arbeitende Prozesse entsteht die Herausforderung, dass konkurrierende Zugriffe auf gemeinsame Ressourcen zu inkonsistenten Zuständen führen können. Ohne geeignete Mechanismen können Änderungen verloren gehen, widersprüchliche Zustände entstehen oder mehrere Prozesse dieselben

Daten gleichzeitig verändern (Saake, Heuer und Sattler 2018, Kap. 13.1.2).

Im Kontext von Background Job-Processing tritt dieses Problem besonders dann auf, wenn mehrere Worker-Instanzen parallel auf denselben Jobbestand zugreifen. Versuchen sie ohne geeignete Synchronisationsmechanismen denselben Job zu beanspruchen oder konkurrierende Statusänderungen durchzuführen, können Race Conditions entstehen. Insbesondere bei Pull-basierten Architekturen, bei denen die Jobverteilung nicht durch eine Orchestrierungskomponente realisiert wird, muss eine solche ungewollte Doppelbeanspruchung explizit verhindert werden (Kleppmann 2017, Kap. 7 Abschnitt „Isolation“). Die Gewährleistung konsistenter Zustände stellt daher eine zentrale Anforderung an Systeme mit paralleler Verarbeitung dar. Die hierfür etablierten Konzepte und Mechanismen werden in den folgenden Abschnitten vorgestellt.

2.2.1 ACID

Zur Vereinfachung der Gewährleistung konsistenter Zustände bei parallelen Zugriffen existiert das Konzept der Transaktionen. Diese fassen mehrere Operationen zu einer logischen Einheit zusammen, deren Ausführung nach außen als zusammenhängender Vorgang erscheint (Kleppmann 2017, Kap. 7). Im Kontext persistenter Datenhaltung werden von Transaktionen typischerweise die vier zentralen Eigenschaften Atomicity, Consistency, Isolation, Durability (ACID) gefordert (Saake, Heuer und Sattler 2018, Kap. 13.1.1):

Atomicity Die Atomarität stellt sicher, dass eine Transaktion entweder vollständig oder gar nicht ausgeführt wird. Tritt während der Ausführung ein Fehler auf, werden sämtliche bis dahin vorgenommenen Änderungen verworfen, sodass keine partiellen Zwischenzustände bestehen bleiben.

Consistency Die Konsistenz beschreibt die Eigenschaft, dass eine erfolgreich abgeschlossene Transaktion die Daten von einem gültigen Zustand in einen anderen gültigen Zustand überführt. Dabei müssen definierte Integritätsbedingungen erhalten bleiben. Kleppmann (2017, Kap. 7 Abschnitt „Consistency“) weist darauf hin, dass die Einhaltung solcher fachlichen Invarianten in der Verantwortung der Anwendung liegt und nicht wie die anderen Eigenschaften durch das Datenbanksystem garantiert werden kann.

Isolation Die Isolation gewährleistet, dass parallel ausgeführte Transaktionen sich nicht gegenseitig beeinflussen. Aus Sicht einer Transaktion soll die Ausführung so erscheinen, als würde sie allein auf dem Datenbestand arbeiten. Dadurch können Race Conditions und andere Nebenläufigkeitsprobleme vermieden werden.

Durability Die Dauerhaftigkeit garantiert, dass die Ergebnisse einer erfolgreich abgeschlossenen Transaktion auch nach Systemabstürzen oder anderen Fehlern dauerhaft erhalten bleiben.

Im Kontext von Background Job-Processing sind insbesondere die Eigenschaften der Atomarität und Isolation von Bedeutung. Während die Atomarität verhindert, dass Statusänderungen eines Jobs nur teilweise durchgeführt werden, bildet die Isolation die

Grundlage für die koordinierte Verarbeitung gemeinsamer Datenbestände durch mehrere parallel arbeitende Worker-Instanzen. Die relevanten Konzepte zum Erreichen dieser Eigenschaften werden in den folgenden Abschnitten näher betrachtet.

2.2.2 Transaktionsgrenzen

Während ACID die theoretischen Garantien einer Transaktion definiert, bestimmt die Festlegung von Transaktionsgrenzen, welche konkreten Datenbankoperationen des softwareseitigen Kontrollflusses zu einer unteilbaren, logischen Einheit zusammengefasst werden. Die Festlegung dieser Grenzen steuert, wann eine Datenbanksitzung eine Transaktion initiiert und zu welchem Zeitpunkt die Änderungen final festgeschrieben (Commit) oder im Fehlerfall vollständig verworfen (Rollback) werden (Kudraß 2015, S.250).

Die Wahl geeigneter Transaktionsgrenzen wird zusätzlich dadurch erschwert, dass fachliche und technische Transaktionen nicht zwangsläufig identisch sind. Eine fachliche Transaktion beschreibt eine zusammenhängende Geschäftsoperation, deren Ergebnis aus Sicht der Anwendung als Einheit betrachtet wird. Eine technische Transaktion hingegen bezeichnet die konkrete Transaktion auf der Datenbankebene, welche die ACID-Eigenschaften für die von ihr gekapselten Datenbankoperationen gewährleistet. Während eine fachliche Operation mehrere technische Transaktionen umfassen kann, können innerhalb einer technischen Transaktion auch nur Teilaspekte einer fachlichen Operation verarbeitet werden. Werden die Grenzen falsch gesetzt, können verschiedene Parallelitätsanomalien auftreten (Saake, Heuer und Sattler 2018, Kap. 13.1.2). Ein verbreiteter Ansatz zur Vermeidung dieser Anomalien ist die Verwendung von Sperren, auf die in Kapitel 2.2.3 näher eingegangen wird. Wichtig ist an dieser Stelle, dass die gewählten Transaktionsgrenzen direkten Einfluss auf die Dauer von Sperren und somit auch auf parallel ausgeführte Transaktionen haben.

Eine Möglichkeit besteht darin, jede Datenbankoperation in einer eigenen Transaktion auszuführen. Dadurch werden Sperren nur für sehr kurze Zeit gehalten, was die Parallelität und den Durchsatz des Systems erhöht. Allerdings können zusammengehörige Änderungen nicht mehr atomar ausgeführt werden. Tritt zwischen mehreren aufeinanderfolgenden Operationen ein Fehler auf, können bereits festgeschriebene Änderungen nicht mehr gemeinsam zurückgesetzt werden. Die betroffenen Daten können dadurch in einen fachlich falschen Zustand überführt werden.

Als naiver Gegenentwurf können die Transaktionsgrenzen entsprechend der fachlichen Transaktion gesetzt werden. Diese Strategie kann jedoch zu langlaufenden Transaktionen führen, welche die Leistungsfähigkeit des Datenspeichers beeinträchtigt. Zum einen halten Transaktionen im Beispiel des 2-Phasen-Sperrprotokolls sämtliche während ihrer Laufzeit akquirierten Sperren bis zum endgültigen Commit oder Rollback aufrecht (Kleppmann 2017, Kap. 7 Abschnitt „Two-Phase Locking (2PL)“). Dadurch steigt die Wahrscheinlichkeit, dass konkurrierende Transaktionen aufeinander warten müssen. Zum anderen können langlaufende Transaktionen die Bereinigung alter Datenversionen verzögern, da diese weiterhin für aktive Transaktionen sichtbar bleiben müssen (The PostgreSQL Global Development Group (Hrsg.) 2026, Kap. 24.1.2). Dies erhöht den Ressourcenbedarf des Datenbanksystems und kann dessen Leistungsfähigkeit beeinträchtigen.

Im Software-Entwurf erfordert das Setzen der Transaktionsgrenzen daher eine kontinuierliche Abwägung zwischen der Systemperformance und dem Schutz vor Datenanomalien. Weit gefasste Transaktionsgrenzen reduziert das Risiko der Entstehung von Inkonsistenzen, blockieren jedoch Systemressourcen und Datensätze. Eng gefasste Grenzen erhöhen den Durchsatz, können jedoch das Risiko konkurrierender Zugriffe und daraus resultierender Race Conditions erhöhen.

2.2.3 Isolationsstufen und Synchronisationsverfahren

Zur Umsetzung der Isolationseigenschaft von Transaktionen müssen konkurrierende Zugriffe auf gemeinsame Daten koordiniert werden. Zu diesem Zweck stellen Datenbanksysteme unterschiedliche Isolationsstufen bereit. Diese schwächen Teile der ACID-Garantien bewusst ab und legen damit fest, in welchem Umfang Transaktionen die Änderungen parallel laufender Transaktionen beobachten und welche Parallelitätsanomalien dadurch im System auftreten können (Kudraß 2015, S. 251). Je nach Anwendungskontext kann es sein, dass nicht alle dieser Anomalien auftreten können, beispielsweise wenn nur Leseoperationen ausgeführt werden, oder dass deren Auftreten in Kauf genommen werden kann, um eine bessere Performance zu erhalten. Dadurch müssen weniger Überprüfungen stattfinden und weniger Transaktionen müssen zurückgesetzt werden, was zu einer besseren Performance führt. Höhere Isolationsstufen bieten stärkere Konsistenzgarantien, können jedoch die Parallelität des Systems verringern (Saake, Heuer und Sattler 2018, Kap. 13.1.3). Während die Isolationsstufen somit die Konsistenzgarantien definieren, kommen zur Koordination der Transaktionen im Datenbanksystem verschiedene Synchronisationsverfahren zum Einsatz, die durch sperrende oder nicht-sperrende Strategien das Entstehen nicht zugelassener Anomalien verhindern (Kudraß 2015, S. 253–259).

Eine verbreitete Strategie zur Durchsetzung der Isolation ist die pessimistische Synchronisation. Sie basiert auf der Annahme, dass konkurrierende Zugriffe auftreten können und verhindert diese bereits im Vorfeld durch Sperrmechanismen. Bevor eine Transaktion einen Datensatz verändert, wird dieser für andere konkurrierende Transaktionen gesperrt. Der Vorteil pessimistischer Verfahren besteht darin, dass Konflikte bereits vor ihrer Entstehung verhindert werden. Dadurch eignen sie sich insbesondere für Szenarien mit häufigen konkurrierenden Zugriffen. Nachteilig wirken sich jedoch zusätzliche Wartezeiten aus, da Transaktionen auf die Freigabe benötigter Sperren warten müssen. Bei hoher Auslastung kann dies die Parallelität und den Gesamtdurchsatz eines Systems reduzieren. Außerdem entsteht die Gefahr von Deadlocks (Kudraß 2015, S. 253–257).

Alternative Ansätze sind nicht sperrende Verfahren, beispielsweise die optimistische Synchronisation oder das Zeitstempelverfahren. Anstatt Daten vor der Bearbeitung zu sperren, werden bei der optimistischen Synchronisation Transaktionen zunächst ausgeführt und erst beim Schreiben geprüft, ob ein Konflikt entstanden ist. Ist es zwischen zwei oder mehreren Transaktionen zum Konflikt gekommen, müssen einzelne Transaktionen zurückgesetzt und wiederholt werden. Bei dem Zeitstempelverfahren werden Zeitstempel für jede Transaktionen, sowie der Zeitpunkt des letzten Lese- und Schreibzugriffs für jedes Datenbankobjekt verwaltet. Anhand dieser Zeitstempel kann geprüft werden, ob zwei

Transaktionen in Konflikt zueinander stehen und ob eine der Transaktionen zurückgesetzt werden muss. Nicht sperrende Verfahren vermeiden die durch Sperren verursachten Wartezeiten und ermöglichen dadurch eine hohe Parallelität. Treten jedoch Konflikte auf, müssen betroffene Operationen wiederholt werden. Sie eignen sich daher insbesondere für Systeme, in denen konkurrierende Änderungen selten vorkommen (Kudraß 2015, S. 257–259).

Welche Synchronisationsstrategie geeignet ist, hängt von den Anforderungen des jeweiligen Systems und der Häufigkeit erwarteter Konflikte ab. Während pessimistische Verfahren Konflikte präventiv verhindern, akzeptieren optimistische Verfahren mögliche Konflikte und behandeln diese nachträglich.

2.3 Fehlertoleranz und Wiederherstellung

Verteilte Softwaresysteme sind zwangsläufig mit einer Vielzahl potenzieller Fehlerquellen konfrontiert. Prozesse können unerwartet terminieren, Netzwerkverbindungen unterbrochen werden oder externe Dienste zeitweise nicht erreichbar sein (Newman 2026, Kap. 1 Abschnitt „The Three Golden Rules Of Distributed Systems“). Beispielsweise kann es bei einzelnen Teilausfällen dazu kommen, dass unklar bleibt, ob ein Verarbeitungsschritt erfolgreich abgeschlossen wurde oder erneut ausgeführt werden muss. Da sich solche Fehler in komplexen Systemlandschaften nicht vollständig vermeiden lassen, muss ihre Existenz als grundlegende Systemeigenschaft betrachtet werden (Newman 2026, Kap. 1 Abschnitt „Failure Is Inevitable“).

Die Zuverlässigkeit eines Systems ergibt sich daher nicht primär daraus, Fehler vollständig zu verhindern, sondern daraus, wie gut mit ihrem Auftreten umgegangen wird. Fehlertoleranz bezeichnet in diesem Zusammenhang die Fähigkeit eines Systems, seine wesentlichen Funktionen trotz auftretender Störungen weiterhin bereitzustellen. Darüber hinaus muss ein System in der Lage sein, nach einem Fehler einen konsistenten Zustand wiederherzustellen und die Verarbeitung kontrolliert fortzusetzen, beziehungsweise neu-zustarten (Newman 2026, Kap. 1 „What Is Resilience?“). Die folgenden Abschnitte stellen die hierfür relevanten Konzepte und Mechanismen vor.

2.3.1 Fehlerklassifikation

Da Fehler in verteilten Systemen unvermeidbar sind, stellt sich nicht die Frage, ob sie auftreten, sondern wie mit ihnen umgegangen werden soll. Die Wirksamkeit von Fehlertoleranzmechanismen hängt dabei wesentlich davon ab, die Ursache und Eigenschaften eines Fehlers korrekt einzuordnen. Nicht jeder Fehler kann durch dieselben Maßnahmen behandelt werden. Während einige Fehler durch eine erneute Ausführung einer Operation behoben werden können, sind andere dauerhaft und erfordern eine Anpassung der Eingabedaten oder eine manuelle Intervention. Die Klassifikation von Fehlern bildet daher die Grundlage für Entscheidungen über Retry-Strategien, Fehlerbehandlung und Wiederherstellungsmechanismen (Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“).

Darüber hinaus können nicht alle denkbaren Fehler mit vertretbarem Aufwand behandelt werden. Während kurzzeitige Infrastrukturprobleme, Prozessabstürze oder Kommunikationsfehler häufig durch geeignete Architekturmechanismen abgefangen werden können, liegen andere Ereignisse, etwa großflächige Naturkatastrophen oder der vollständige Ausfall eines Rechenzentrums, gegebenenfalls außerhalb des Fehlerannahmemodells einer Anwendung. Der Entwurf fehlertoleranter Systeme erfordert daher eine bewusste Entscheidung darüber, welche Fehlerarten berücksichtigt werden und welche Risiken akzeptiert werden müssen (Kleppmann 2017, Kap. 1 Abschnitt „Reliability“).

Eine grundlegende Einteilung erfolgt in transiente und permanente Fehler. Transiente Fehler treten zeitlich begrenzt auf und verschwinden nach kurzer Zeit wieder. Ursachen liegen häufig in der technischen Infrastruktur eines Systems, beispielsweise in vorübergehenden Netzwerkstörungen, kurzzeitigen Überlastungen von Diensten, nicht erreichbaren Datenbanken oder dem Neustart einzelner Systemkomponenten (Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“).

Permanente Fehler hingegen bestehen unabhängig von der Anzahl der Wiederholungsversuche fort. Ihre Ursache liegt häufig in der Anwendung selbst oder in den zu verarbeitenden Daten. Beispiele hierfür sind ungültige Eingabedaten oder fehlende Berechtigungen. Newman (2026, Kap. 3 Abschnitt „Should You Always Retry?“) verdeutlicht dies anhand von HTTP-Fehlercodes wie 404 Not Found oder 403 Forbidden.

Die Abgrenzung zwischen beiden Fehlerarten ist insbesondere für die automatisierte Fehlerbehandlung relevant. Während transiente Fehler häufig eine erneute Ausführung rechtfertigen, sollten permanente Fehler möglichst früh erkannt und gesondert behandelt werden.

Eine besondere Rolle spielen zudem Timeout-Fehler. In verteilten Systemen werden Timeouts verwendet, um potenzielle Ausfälle zu erkennen. Dabei kann jedoch nicht eindeutig unterschieden werden, ob tatsächlich ein Fehler vorliegt oder die Antwort lediglich ungewöhnlich lange braucht. Ein Timeout signalisiert daher nur, dass eine erwartete Antwort nicht innerhalb eines definierten Zeitfensters eingetroffen ist und lässt somit keine Aussage über eine Ursache zu (Bass 2021, Kap. 17.2 Abschnitt „Failure in the Cloud“).

2.3.2 Retry-/Backoff-Strategien

Die in Kapitel 2.3.1 eingeführte Unterscheidung zwischen transienten und permanenten Fehlern bildet die Grundlage für den Einsatz von Wiederholungsmechanismen. Da transiente Fehler nur zeitweise die Verarbeitung behindern, kann in solchen Fällen eine erneute Ausführung einer fehlgeschlagenen Operation erfolgreich sein, obwohl der ursprüngliche Versuch fehlgeschlagen ist (Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“). Retry-Strategien verfolgen damit also das Ziel, die Erfolgswahrscheinlichkeit einer Verarbeitung zu erhöhen, ohne dass ein manueller Eingriff erforderlich wird. Retries sollten ausschließlich für Fehler durchgeführt werden, deren Ursache außerhalb der eigentlichen Geschäftslogik liegt und deren Fortbestand nicht dauerhaft erwartet wird. Permanente Fehler sollten dagegen unmittelbar als endgültig fehlgeschlagen behandelt werden, da eine erneute Ausführung die Erfolgswahrscheinlichkeit nicht erhöht. Ein typisches Szenario, in dem Retries sinnvoll sind, ist das Verschicken von Nachrichten

(Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“; Microsoft Corporation (Hrsg.) 2026a, Abschnitt „Reliability considerations“).

Ein mögliches Vorgehen bei einem temporären Fehler ist in Abbildung 2.3 dargestellt. Da die Ursache eines transienten Fehlers potenziell auch für eine gewisse Zeit nach dem Auftreten des Fehlers weiterhin besteht, werden Wiederholungen üblicherweise nicht unmittelbar durchgeführt. Stattdessen wird zwischen den einzelnen Versuchen eine Wartezeit eingefügt. Dieses Vorgehen wird als Backoff bezeichnet und soll verhindern, dass wiederholte Fehlversuche zusätzliche Last auf bereits beeinträchtigte Systeme erzeugen. Bekommt eine Anfrage beispielsweise ein Timeout, weil der Server zu viele Anfragen erhält, würden direkte Retries den Server noch weiter mit Anfragen verstopfen. Eine einfache Variante besteht darin, zwischen allen Wiederholungsversuchen dieselbe Wartezeit zu verwenden. In verteilten Systemen wird jedoch häufig ein exponentieller Backoff eingesetzt, bei dem sich das Warteintervall nach jedem fehlgeschlagenen Versuch schrittweise erhöht. Dadurch erhält das betroffene System mehr Zeit, sich von einer temporären Störung zu erholen, da die Dichte der Wiederholungsversuche reduziert wird (Newman 2026, Kap. 3 Abschnitt „Delays Between Retries“).

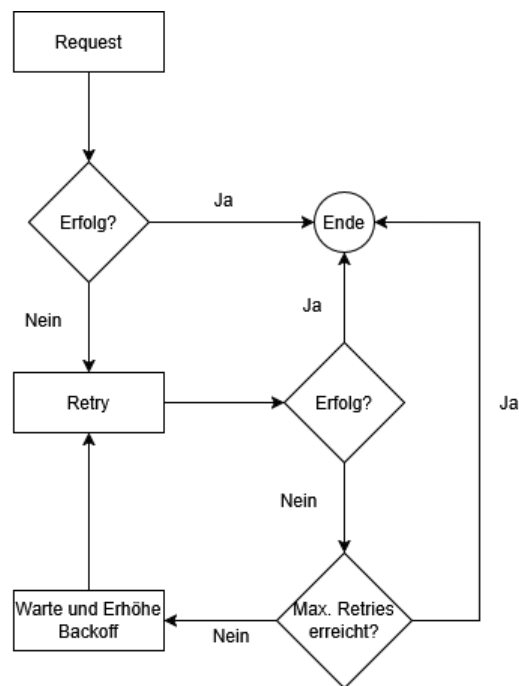


Abbildung 2.3: Verhalten bei transienten Fehlern

Neben der zeitlichen Staffelung von Wiederholungsversuchen muss auch deren Anzahl begrenzt werden. Andernfalls besteht die Gefahr, dass ein fehlerhafter Job unendlich oft ausgeführt wird und dadurch dauerhaft Ressourcen belegt. In der Praxis wird daher eine maximale Anzahl von Wiederholungsversuchen definiert. Wird diese überschritten, gilt die Verarbeitung als endgültig fehlgeschlagen und der Job wird in einen entsprechen-

den Fehlerzustand überführt (Newman 2026, Kap. 3 Abschnitt „How Many Retries Are Appropriate?“).

Retry- und Backoff-Strategien bilden damit einen wesentlichen Baustein zur Behandlung transienter Fehler. Sie ermöglichen die automatische Wiederholung fehlgeschlagener Verarbeitungen, ohne abhängige Systeme durch unmittelbar aufeinanderfolgende Wiederholungsversuche zusätzlich zu belasten.

2.3.3 Recovery

Neben transienten Fehlern, die häufig durch Retry-Mechanismen behandelt werden können, die durch Retries gelöst werden können, können auch Fehler auftreten, die den Ausführungsprozess einer Verarbeitung vollständig unterbrechen. Besondere Relevanz besitzen Ausfälle ausführenden Komponenten während einer laufenden Bearbeitung. In einem solchen Szenario kann kein lokaler Retry mehr initiiert werden, da die ausführende Komponente selbst nicht mehr verfügbar ist. Recovery bezeichnet in diesem Kontext die Fähigkeit eines Systems, eine derart unterbrochene Verarbeitung zu erkennen und geeignete Maßnahmen zur Wiederaufnahme einzuleiten (Bass 2021, Kap. 4.2).

Die Notwendigkeit solcher Mechanismen ergibt sich aus den Eigenschaften verteilter Systeme. Da einzelne Komponenten unabhängig voneinander ausfallen können, während andere Teile des Systems weiterhin funktionieren, entstehen sogenannte Teilausfälle (Kleppmann 2017, Kap. 8 Abschnitt „Faults and Partial Failures“). Dadurch entsteht Unsicherheit über den tatsächlichen Verarbeitungszustand einer Aufgabe. Gegebenenfalls ist nicht eindeutig feststellbar, ob eine Verarbeitung bereits erfolgreich abgeschlossen wurde aber die Bestätigung darüber fehlt, kurz vor ihrem Abschluss steht oder aufgrund eines Fehlers vorzeitig abgebrochen wurde (Kleppmann 2017, Kap. 8).

Zur Wiederherstellung nach einem Ausfall muss zwischen flüchtigem und persistentem Zustand unterschieden werden. Während lokal im Arbeitsspeicher gehaltene Informationen beim Absturz eines Prozesses verloren gehen, kann persistent gespeicherter Zustand auch nach einem Ausfall weiterhin genutzt werden (Kleppmann 2017, Kap. 11 Abschnitt „Rebuilding state after a failure“). Die Verarbeitung auf Basis eines Zwischenzustands wieder aufzunehmen ist unter Umständen jedoch nicht trivial. Eine verbreitete Wiederherstellungsstrategie besteht daher darin, die für eine Verarbeitung relevanten Eingabedaten dauerhaft zu speichern und eine unterbrochene Verarbeitung bei Bedarf auf Basis dieser Ursprungsdaten erneut auszuführen. Dies ist häufig einfacher und robuster als der Versuch, den exakten Laufzeitzustand eines abgestürzten Prozesses zu rekonstruieren und die Verarbeitung dort fortzusetzen (Bass 2021, Kap. 17.3). Eine erneute Ausführung setzt voraus, dass die zur Verarbeitung benötigten Informationen auch nach einem Ausfall weiterhin verfügbar sind. Darüber hinaus darf ein Ausfall nicht zu inkonsistenten Zuständen führen, die eine spätere Wiederaufnahme verhindern. Recovery-Mechanismen werden daher häufig mit Verfahren kombiniert, die eine konsistente Speicherung relevanter Zustandsinformationen gewährleisten (Bass 2021, Kap. 4.2 Abschnitt „Recover from Faults“).

Voraussetzung für eine Wiederaufnahme ist zunächst die Erkennung unterbrochener Verarbeitungen. Wird eine Verarbeitung nach ihrer Übernahme durch eine ausführende

Komponente unterbrochen, kann ein Zustand entstehen, in dem die Aufgabe aus Sicht des Systems weiterhin als aktiv gilt, obwohl tatsächlich keine Bearbeitung mehr stattfindet. Zur Erkennung solcher Situationen werden häufig Heartbeat- oder Lease-/Timeout-Mechanismen eingesetzt (Bass 2021, Kap. 17.2 Abschnitt „Failure in the Cloud“). Hierbei sendet die ausführende Komponente in regelmäßigen Abständen Signale, anhand derer ihre Aktivität überwacht werden kann. Bleiben diese Signale über einen definierten Zeitraum aus, wird ein Ausfall vermutet und die Verarbeitung als potenziell unterbrochen betrachtet (Bass 2021, Kap. 4.2 Abschnitt „Detect Faults“).

Nach der Erkennung eines Ausfalls muss die Verarbeitung erneut angestoßen werden. Hierfür wird die betroffene Aufgabe erneut zur Bearbeitung bereitgestellt, indem sie wieder freigegeben wird. Da eine Verarbeitung häufig erst nach einer expliziten Erfolgsquittierung als abgeschlossen gilt, führt das Ausbleiben einer solchen Quittung infolge eines Ausfalls dazu, dass die Aufgabe erneut verarbeitet wird (Microsoft Corporation (Hrsg.) 2026a).

Für Recovery sind also zwei Schritte essenziell. Zunächst muss erkannt werden, dass eine Verarbeitung unterbrochen wurde. Anschließend muss die Verarbeitung neu angestoßen werden. Dafür existieren unterschiedliche Ansätze. Einerseits kann ein System Zwischenzustände persistent speichern und die Verarbeitung nach einem Ausfall auf Basis des zuletzt bekannten Zustands fortsetzen. Andererseits kann die Verarbeitung vollständig neu gestartet werden, sofern die dafür benötigten Eingabedaten weiterhin verfügbar sind. Welcher Ansatz gewählt wird, hängt unter anderem von der Komplexität der Verarbeitung sowie vom Aufwand einer vollständigen Neuausführung ab.

Die in diesem Kapitel beschriebene Wiederaufnahme führt zwangsläufig dazu, dass einzelne Verarbeitungsschritte mehrfach ausgeführt werden können. Da dabei häufig nicht eindeutig festgestellt werden kann, ob eine vorherige Ausführung bereits teilweise oder vollständig erfolgreich war, müssen Systeme auf mögliche Mehrfachausführungen vorbereitet sein. Hieraus ergibt sich die Notwendigkeit idempotenter Verarbeitung, die im folgenden Abschnitt näher betrachtet wird.

2.3.4 Idempotenz

Die in den vorherigen Abschnitten beschriebenen Retry- und Wiederherstellungsmechanismen erhöhen die Zuverlässigkeit eines Systems, führen jedoch gleichzeitig dazu, dass einzelne Verarbeitungsschritte mehrfach ausgeführt werden können. Um trotz solcher Mehrfachausführungen konsistente Datenbestände sicherzustellen, wird das Konzept der Idempotenz eingesetzt. Eine Operation wird als idempotent bezeichnet, wenn ihre mehrmalige Ausführung denselben fachlichen Zustand erzeugt wie eine einmalige Ausführung (Newman 2026, Kap. 3 Abschnitt „Idempotency“). Wiederholte Aufrufe verändern das Ergebnis somit nicht, wodurch Duplikate oder erneut ausgeführte Verarbeitungsschritte keine inkonsistenten Zustände verursachen. Wie in Kapitel 2.1.2 dargelegt, stellt Idempotenz damit eine wesentliche Voraussetzung dar, um auf Basis einer At-Least-Once-Semantik eine Exactly-Once-Semantik zu erreichen.

Idempotenz kann dabei auf unterschiedlichen Ebenen eines Systems umgesetzt werden. In Bezug auf Job-Processing muss bereits die Aufnahme eines Jobs in den Job-Store

idempotent gestaltet werden, um zu verhindern, dass derselbe Job mehrfach im System erfasst wird. Ein verbreiteter Ansatz zur idempotenten Annahme von Anfragen ist die Implementierung eines idempotenten Receivers (Hohpe und Woolf 2005, Kap. 10 Abschnitt „Idempotent Receiver“) unter Verwendung eindeutiger Identifikatoren, wie beispielsweise eines Universally Unique Identifier (UUID). Bei einer möglichen Variante wird jeder Anfrage vom Sender eine eindeutige Kennung zugefügt, die vom Empfänger zusammen mit der Anfrage persistent gespeichert wird. Geht dieselbe Anfrage erneut ein, kann das System anhand dieser UUID erkennen, dass die Anfrage bereits verarbeitet wurde und eine redundante Speicherung verhindern (Newman 2026, Kap. 3 Abschnitt „Client Request IDs“; Richardson 2018, Kap. 3.3.6).

Darüber hinaus muss auch die eigentliche Verarbeitung eines Jobs so gestaltet werden, dass wiederholte Ausführungen und Wiederaufnahmen keine inkonsistenten Ergebnisse erzeugen (Newman 2026, Kap. 3 Abschnitt „Idempotency“). Statusänderungen und fachliche Datenänderungen müssen folglich über klar definierte Transaktionsgrenzen abgesichert werden, um eine konsistente Weiterbearbeitung nach einem Crash zu gewährleisten (Kleppmann 2017, Kap. 7).

Idempotenz bildet somit die Grundlage dafür, dass Retry- und Recovery-Mechanismen eingesetzt werden können, ohne die Konsistenz der verarbeiteten Daten zu gefährden. Sie stellt einen zentralen Baustein fehlertoleranter Background-Job-Processing-Systeme dar und ermöglicht die korrekte Verarbeitung trotz unvermeidbarer Mehrfachausführungen und Duplikate.

2.4 Stand der Forschung

Die Verarbeitung von Hintergrundaufgaben ist ein etabliertes Problemfeld verteilter Systeme. Entsprechend existieren zahlreiche Frameworks und Architekturmuster, die eine asynchrone Hintergrundaufführung von Aufgaben ermöglichen. Die Ansätze unterscheiden sich insbesondere hinsichtlich der verwendeten Infrastruktur, der Persistenzmechanismen sowie der Umsetzung von Fehlertoleranz und Wiederherstellung. Im Folgenden werden die für diese Arbeit relevanten Lösungsansätze vorgestellt und hinsichtlich ihrer Eigenschaften eingeordnet.

2.4.1 Brokerbasierte Ansätze

Eine weit verbreitete Klasse von Background Job-Processing-Systemen basiert auf dedizierten Message Brokern wie RabbitMQ (Broadcom Inc. (Hrsg.) 2026a) oder Apache Kafka (Apache Software Foundation (Hrsg.) 2026). Hierbei werden Aufgaben in Form von Nachrichten an einen Broker übergeben, der die Entkopplung zwischen Produzenten und Konsumenten übernimmt. Worker beziehen die Nachrichten über den Broker und führen die zugehörige Verarbeitung aus. Der Broker läuft als eigenständige Middleware-Komponente entweder nativ oder virtuell als Server. Der Vorteil dieses Ansatzes liegt in der hohen Skalierbarkeit und der klaren Trennung zwischen Anwendungslogik und Nachrichteninfrastruktur. Darüber hinaus stellen moderne Broker Mechanismen zur Fehlertoleranz, Lastverteilung und Ausfallsicherheit bereit. Gleichzeitig entsteht durch broker-

basierte Architekturen zusätzliche betriebliche Komplexität, da neben der eigentlichen Anwendung weitere Infrastrukturkomponenten aufgesetzt, betrieben und überwacht werden müssen. Des Weiteren ergeben sich Herausforderungen bei der Konsistenz zwischen Anwendungsdatenbank und Message Broker. Problematisch ist beispielsweise die atomare Kopplung von Datenänderungen in der Datenbank und das Verschicken einer Nachricht. Da beide Systeme unabhängig voneinander arbeiten, müssen zusätzliche Mechanismen wie das Transactional Outbox Pattern eingesetzt werden, um Nachrichtenverlust oder Inkonsistenzen zu vermeiden (Richardson 2018, Kap. 3.3.7).

2.4.2 Datenbankbasierte Ansätze

Neben brokerbasierten Lösungen existieren Frameworks, die eine relationale Datenbank als zentralen Job-Store verwenden. Zu bekannten Vertretern dieses Ansatzes zählen unter anderem JobRunr (Rosoco BV (Hrsg.) 2026), Hangfire (Hangfire OÜ (Hrsg.) 2026) oder Spring Batch (Broadcom Inc. (Hrsg.) 2026b). Anstatt Aufgaben an einen separaten Broker zu übertragen, werden diese als persistente Datensätze in einer Datenbank gespeichert und von Workern direkt ausgelesen.

Die Nutzung einer relationalen Datenbank ermöglicht den Gebrauch der ACID-Eigenschaften, um das Speichern neuer Jobs, Verwalten von Status- und fachlichen Datenänderungen konsistent innerhalb einer gemeinsamen Transaktion zu verarbeiten. In der Praxis verfolgen existierende Frameworks jedoch unterschiedliche Schwerpunkte und adressieren die Problemstellung jeweils aus einer anderen Perspektive. Hangfire garantiert nur eine At-Least-Once Ausführung und überlässt die Umsetzung von Idempotenz der Fachlogik. Während der Quartz Scheduler (Terracotta Inc. (Hrsg.) 2026) primär auf zeitgesteuertes Scheduling und clusteringbasierte Koordination fokussiert, stellt Spring Batch eine spezialisierte Infrastruktur für die blockweise Massenverarbeitung bereit, die auf periodische oder zeitgesteuerte Durchläufe ausgelegt ist. Modernere Java-Bibliotheken wie JobRunr und Hangfire integrieren zwar direkt ein automatisiertes Retry-Handling, kapseln die zugrundeliegenden Mechanismen jedoch als Blackbox, wodurch eine feingranulare Kontrolle und die präzise Analyse des Systemverhaltens bei Infrastrukturausfällen erschwert werden. Robuste und langlebige Workflow-Ausführungen wie Temporal (Temporal Technologies Inc. 2026) lösen die Frage nach Fehlertoleranz wiederum über eine dedizierte Engine, erzwingen damit jedoch einen eigenständigen Serverbetrieb und eine damit einhergehende infrastrukturelle Komplexität. Die konkrete Umsetzung der Nebenläufigkeitskontrolle, Wiederherstellung und Fehlerbehandlung ist stark von den jeweiligen Frameworks abhängig. Die zugrundeliegenden Prinzipien bleiben somit hinter der Schnittstelle der Frameworks verborgen und lassen sich nicht ohne Weiteres abstrahieren oder auf eigene Systementwürfe übertragen.

2.4.3 Einordnung und Forschungslücke

Die vorgestellten Ansätze zeigen, dass sowohl brokerbasierte als auch datenbankbasierte Architekturen zur Umsetzung von Background-Job-Processing-Systemen geeignet sind. Während bestehende Frameworks bereits umfangreiche Funktionalitäten für Scheduling,

Retry-Mechanismen und Fehlertoleranz bereitstellen, liegt ihr Fokus primär auf der praktischen Nutzung, wodurch die zugrundeliegenden transaktionalen Mechanismen und deren Verhalten in kritischen Ausfallszenarien für den Entwickler oft intransparent bleiben. Insbesondere die Frage, unter welchen Voraussetzungen eine relationale Datenbank gleichzeitig als persistenter Job-Store und Koordinationsmechanismus eingesetzt werden kann, wird in der Literatur nur begrenzt behandelt. Konzepte wie Fehlertoleranz, Recovery, Idempotenz und Nebenläufigkeitskontrolle werden zwar häufig einzeln beschrieben, ihre Integration zu einer konsistenten Gesamtarchitektur bleibt jedoch oftmals implizit.

Die vorliegende Arbeit grenzt sich von bestehenden Frameworks ab, rein datenbankgestütztes Pull-Modell entwirft und bezüglich der angestrebten Fehlertoleranz evaluiert. Durch den Einsatz von SQL-Polling in Kombination mit PostgreSQL-Funktionalität **FOR UPDATE SKIP LOCKED** und einem Statusmodell wird untersucht, wie die Koordination und Ausfallwiederherstellung direkt auf der Datenbankebene orchestriert werden kann, ohne einen separaten Message Broker einzusetzen.

3 Methodik

3.1 Literaturrecherche

3.2 Ergebnisgenerierung

4 Anforderungen

„Darüber hinaus können nicht alle denkbaren Fehler mit vertretbarem Aufwand behandelt werden. Während kurzzeitige Infrastrukturprobleme, Prozessabstürze oder Kommunikationsfehler häufig durch geeignete Architekturmechanismen abgefangen werden können, liegen andere Ereignisse, etwa großflächige Naturkatastrophen oder der vollständige Ausfall eines Rechenzentrums, außerhalb des typischen Fehlerannahmemodells vieler Anwendungen. Der Entwurf fehlertoleranter Systeme erfordert daher eine bewusste Entscheidung darüber, welche Fehlerarten berücksichtigt werden und welche Risiken akzeptiert werden müssen.“ Kapitel 2.3.1 -> Begründen, welche Art von Fehlern in meiner Arbeit behandelt werden sollen und wieso ausgerechnet diese

4.1 Funktionale Anforderungen

4.2 Qualitätsanforderungen

4.3 Zusicherungen

Z.B.: Für die Evaluation werden folgende Zusicherungen definiert, die das System zu jedem Zeitpunkt einhalten muss: Z1: Kein Job erreicht mehr als einmal den Status SUCCEEDED. Z2: Zu einem gegebenen Auftrag (Idempotency-Key) wird höchstens ein Report erzeugt. Z3: Statusübergänge folgen ausschließlich dem definierten Zustandsmodell. Ungültige Übergänge (z. B. direkt von QUEUED zu SUCCEEDED) sind ausgeschlossen. Z4: Kein Job verbleibt dauerhaft in einem Zwischenzustand. Jeder Job erreicht innerhalb einer definierten Frist einen Endzustand (SUCCEEDED oder FAILED).

Begründen, warum ich mich für diese Frameworks entschieden habe, die vorkommen. -> Jede (auch implizite) Entscheidung begründen Alles Begründen!!! Wieso ist das überhaupt relevant? Welches Problem löse ich, wieso habe ich das so gemacht? Wieso genau diese Fehlerszenarien? Begründen wieso nicht andere oder mehr? Gibt es formale Modelle die die Zusicherungen zu jedem Zeitpunkt checken?

5 Umsetzung

5.1 Architektur und Design

Einen Message Consumer Idempotent machen (Richardson 2018, Kap.6.1.6) Statusmodell: Consumers may crash at any time, so it could happen that a broker delivers a message to a consumer but the consumer never processes it, or only partially processes it before crashing. In order to ensure that the message is not lost, message brokers use *acknowledgments*: a client must explicitly tell the broker when it has finished processing a message so that the broker can remove it from the queue." (Kleppmann 2017, Kap. 11, Abschnitt „Acknowledgments and redelivery“)

Obwohl das Queue-Based Load Leveling Pattern in der Praxis häufig mit dedizierten Message Brokern implementiert wird, stellt die Verwendung einer relationalen Datenbank wie PostgreSQL als Job-Store für das vorliegende System eine architektonisch bewusste Entscheidung dar. Durch die Nutzung von PostgreSQL-spezifischen Mechanismen wie SKIP LOCKED in Kombination mit vollen ACID-Transaktionsgarantien lässt sich die von Microsoft (2023) geforderte Lastverteilung realisieren, ohne die für die Fehlertoleranz kritische atomare Verknüpfung von Job-Status und Fachdaten aufzugeben. Es wird in dieser Arbeit also nicht direkt ein Message Broker genutzt, sondern ein eigenes System auf Basis einer relationalen Datenbank umgesetzt, was sich funktional stark an den Prinzipien eines Message Brokers orientiert.

„Es ist anzumerken, dass eine verteilte Konsistenz zwischen einem dedizierten Message Broker und einem relationalen Datenspeicher theoretisch auch über ein Two-Phase-Commit-Protokoll (2PC) realisiert werden kann. In der verteilten Softwarearchitektur gilt 2PC jedoch aufgrund des hohen Koordinations- und Netzwerk-Overheads sowie des inhärent blockierenden Verhaltens bei Koordinator-Ausfällen als performanzkritisch und komplex (vgl. Kleppmann 2017, Kap. 9). Die Entscheidung für eine rein datenbankgestützte Architektur auf Basis von PostgreSQL wurde daher bewusst getroffen, um die operationelle Komplexität zu minimieren und eine atomare Verknüpfung von Job-Status und Fachdaten innerhalb einer einzigen, lokalen ACID-Transaktion ohne verteiltes Konsensprotokoll zu garantieren

Obwohl das kontinuierliche Abfragen eines solchen externen Datenspeichers im Vergleich zu rein lokalen Hauptspeicher-Lösungen Latenzen erzeugt, existiert in der verteilten Architektur kein universeller Idealzustand. Die Wahl des Zustandsspeichers ist stets ein bewusster Trade-off zwischen maximalem Nachrichtendurchsatz und strikten Garantien zur Fehlerbehebung (Kleppmann 2017, Kap. 11, Abschnitt „Rebuilding state after a failure“).

Das Führen eines expliziten Zustandsmodells (z. B. *queued*, *running*) in einer persistenten Datenbank ermöglicht es einer neuen oder neu gestarteten Worker-Instanz, den letzten konsistenten Zustand nach einem Teilausfall direkt auszulesen und die Verarbeitung ohne Datenverlust wiederaufzunehmen.

Um eine Ressourcenerschöpfung durch langlaufende Datenbank-Transaktionen zu verhindern, wird die physische Datenbanksperre auf der Datenbankebene zeitlich von der

eigentlichen Job-Ausführung entkoppelt. Ein Worker nutzt einen exklusiven Sperrmechanismus (wie FOR UPDATE SKIP LOCKED) lediglich für die Millisekunden der atomaren Zustandsänderung eines Jobs von *queued* auf *running*. Sofort nach dem Schreiben des neuen Status wird diese isolierte Datenbank-Transaktion geschlossen und die Zeilensperre freigegeben. Die eigentliche, zeitintensive Fachlogik wird anschließend außerhalb einer offenen Transaktion verarbeitet, während das persistente Zustandsmodell im Datenspeicher parallele Zugriffe anderer Worker verhindert (vgl. Kleppmann 2017, Kap. 11).

Ein naiver Ansatz zur Absicherung eines Hintergrundjobs bestünde darin, die Transaktionsgrenze über den gesamten Lebenszyklus der Job-Ausführung zu spannen. Dies bedeutet, dass eine Transaktion geöffnet wird, sobald ein Worker eine Aufgabe akquiriert, und erst schließt, wenn die vollständige Fachlogik abgearbeitet wurde. In produktiven Systemen führt dieses Vorgehen jedoch zu dem Phänomen der langlaufenden Transaktionen (Long-Running Transactions), welche die Resilienz des Gesamtsystems massiv gefährden. Da Hintergrundprozesse definitionsgemäß zeitintensive oder ressourcenintensive Berechnungen sowie E/A-Operationen kapseln (vgl. Kleppmann 2017, Kap. 10), würde eine offene Transaktion über diesen gesamten Zeitraum hinweg die zugewiesene Datenbankverbindung blockieren. Da relationale Datenbanksysteme wie PostgreSQL Verbindungen über einen limitierten Pool (*Connection Pool*) verwalten, führt eine langanhaltende Belegung durch wenige Worker-Instanzen unweigerlich zur Erschöpfung dieser Ressource (*Connection Starvation*). Infolgedessen können synchrone Endpunkte, die für schnelle Client-Anfragen ebenfalls Verbindungen aus diesem Pool benötigen, keine Datenbankinteraktionen mehr durchführen, was zu weitreichenden Timeouts und einem Systemausfall führt (vgl. Kleppmann 2017, Kap. 7).

Die Transaktionsgrenzen müssen so eng wie möglich gefasst werden, was zu einem dreiphasigen Kontrollfluss im Worker führt:

1. **Phase 1 (Gekapselte Statussynchronisation):** Der Worker öffnet eine dedizierte, extrem kurzlaufende Transaktion. Innerhalb dieser Grenze wird ein freier Job gesucht, über Sperrverfahren geschützt und dessen Status im Job-Store atomar von *queued* auf *running* aktualisiert. Unmittelbar nach dem Schreiben des Status wird die Transaktion committet und die Verbindung an den Pool zurückgegeben.
2. **Phase 2 (Transaktionsfreie Fachlogik):** Die eigentliche Verarbeitung des Jobs (z. B. das Generieren eines Reports oder der Aufruf einer externen HTTP-Schnittstelle) erfolgt vollständig außerhalb einer offenen Datenbanktransaktion. Das persistente Zustandsmodell im Speicher verhindert in dieser Phase, dass parallele Worker Zugriff auf denselben Job erhalten (vgl. Kleppmann 2017, Kap. 11).
3. **Phase 3 (Gekapselte Abschlussquittierung):** Nach erfolgreicher Beendigung oder nach einem kontrollierten Abbruch der Fachlogik öffnet der Worker eine zweite, separate Kurzzeittransaktion, um den finalen Endzustand (z. B. *success* oder *failed*) in den Job-Store zu schreiben und sofort zu committen.

erläutern, wieso pessimistisches Locking anstatt von optimistischen verfahren -> Für

die atomare Reservierung von Jobs ist ein pessimistisches Verfahren vorteilhaft, da konkurrierende Zugriffe bereits zum Zeitpunkt der Jobauswahl verhindert werden können. Dadurch lässt sich sicherstellen, dass ein Job nur von einer Worker-Instanz gleichzeitig verarbeitet wird. It performs badly if there is high contention (many transactions trying to access the same objects), as this leads to a high proportion of transactions needing to abort. If the system is already close to its maximum throughput, the additional transaction load from retried transactions can make performance worse." (Kleppmann 2017, Kap. 7 Abschnitt „Pessimistic versus optimistic concurrency control“)

Idempotente Verarbeitung: - nicht mehrere worker dürfen sich den gleichen job ziehen -> muss synchronisiert werden und ein statusmodell existieren, wobei die reservierung atomar passieren muss, -> relationale datenbank mit strikten acid-garantien/isolation levels - nach absturz eines workers muss eine erneute ausführung zum gleichen ergebnis kommen was die erste ausführung erhalten hätte -> dafür müssen die daten, auf denen die fachliche operation aufbaut unverändert vorliegen -> transaktionales modell, wenn prozess abstürzt dürfen keine halb aktualisierten daten zurück bleiben

Dein „remote datastore“ ist praktisch deine PostgreSQL-Jobtabelle (persistenter Job-Store). Das entspricht dem Ansatz „State remote halten“, was Recovery vereinfacht, aber Laufzeit/Contention beeinflusst. Dein Worker hat vermutlich lokalen, transienten Zustand während der Verarbeitung (in-memory). Der muss nach Crash nicht „rekonstruiert“ werden, sondern du brauchst Mechanismen wie Lease/Timeout + Retry, um Jobs wieder aufzunehmen. „Replay state from input“ entspricht bei dir am ehesten: Job erneut ausführen (at-least-once) – und genau deshalb brauchst du Idempotenz, weil „replay“ bei Side-Effects sonst doppelte Ergebnisse erzeugt. (Kleppmann 2017, Kap.11 Rebuilding a state failure)

5.1.1 Zustandsmodell eines Jobs

5.2 Implementierung

(Hohpe und Woolf 2005, Kap. 5 Abschnitt „Command Message“) (Richardson 2018, Kap. 6.2)

5.2.1 Technologie-Stack

5.2.2 Zentrale Code-Entscheidungen

6 Evaluation

6.1 Fehlerszenarien

6.2 Durchführung

6.3 Ergebnisse

7 Diskussion

7.1 Gewonnene Erkenntnisse

7.2 Einschränkungen und Grenzen der Arbeit

Diese Arbeit betrachtet die Fehlertoleranz ab dem Zeitpunkt, zu dem ein Job persistent im Job-Store vorliegt; die zuverlässige Auftragserzeugung/Client-Kommunikation (z.B. Outbox, Messaging) ist nicht Gegenstand. Consumers may crash at any time, so it could happen that a broker delivers a message to a consumer but the consumer never processes it, or only partially processes it before crashing. In order to ensure that the message is not lost, message brokers use *acknowledgments*: a client must explicitly tell the broker when it has finished processing a message so that the broker can remove it from the queue." (Kleppmann 2017, Kap.11 -> Acknowledgments and redelivery bzw. Message brokers)

REST hat keinen Puffer und ist synchron -> Client und Server muss während der ganzen Kommunikation laufen, damit Kommunikation funktioniert. Weitere Nachteile in Referenc: (Richardson 2018, Kap.3.2.2)

Kein Fokus auf Kommunikation im allgemeinen, weder wie oben genannt die Erstellung von Jobs, als auch die Benachrichtigung bei Fertigstellung. Dafür bräuchte es Transaktionales messaging bspw. mittels transactional outbox pattern (Richardson 2018, Kap.3.3.7) oder asynchrones Messaging (Richardson 2018, Kap.3.4.2)

Es gibt keine perfekte reliability (Kleppmann 2017, Kap. 1, Abschnitt „Reliability“)

Load Leveling umgesetzt?

Performance von DB gegenüber Message Queues

Keine Fehlertoleranz in Bezug auf Datenbeständigkeit (z.B. Festplatte des Job-Stores geht kaputt), Naturkatastrophen (Ganzer Standort fällt aus z.B. wegen Überschwemmung) (Newman 2026, Kap. 9)

Es gibt keine Möglichkeit absolute Garantien zu geben, nur viele Möglichkeiten der Risikoreduktion (Kleppmann 2017, Kap. 7 Abschnitt „The Meaning of ACID“-> Ende)

um performance zu verbessern könnte man verschiedene checkpoints in die verarbeitung einbringen, sodass bei einem Abbrechen einer Task nicht die gesamte Task neu gestartet wird sondern ab einem bestimmten punkt wieder angefangen wird (Microsoft Corporation (Hrsg.) 2026a, Abschnitt „Reliability considerations“).

vorsicht bei dingen bei denen kein rollback gemacht werden kann: versendet ein worker eine email, stürzt ab und die operation wird wiederholt wird potenziell noch eine email versendet. das muss gesondert abgefangen werden

8 Fazit und Ausblick

Literaturverzeichnis

- Apache Software Foundation (Hrsg.) (22. Mai 2026). *Apache Kafka Documentation 4.3.X*. AK 4.3.X. URL: <https://kafka.apache.org/43/> (besucht am 26.06.2026).
- Bass, Len (Aug. 2021). *Software architecture in practice*. Unter Mitarb. von Paul Clements und Rick Kazman. Fourth edition. SEI series in software engineering. Boston: Addison-Wesley.
- Broadcom Inc. (Hrsg.) (15. Juni 2026a). *RabbitMQ Documentation / RabbitMQ*. URL: <https://www.rabbitmq.com/docs> (besucht am 26.06.2026).
- (24. Mai 2026b). *Spring Boot*. Spring Boot. URL: <https://spring.io/projects/spring-boot> (besucht am 24.05.2026).
- Hangfire OÜ (Hrsg.) (26. Juni 2026). *Hangfire – Background jobs and workers for .NET and .NET Core*. URL: <https://www.hangfire.io> (besucht am 26.06.2026).
- Hohpe, Gregor und Bobby Woolf (2005). *Enterprise integration patterns designing, building, and deploying messaging solutions*. 5. print. Boston [u.a.: Addison-Wesley.
- Kleppmann, Martin (2017). *Designing data-intensive applications: the big ideas behind reliable, scalable, and maintainable systems*. First edition. Beijing, [China: O'Reilly.
- Kudraß, Thomas (5. März 2015). *Taschenbuch Datenbanken*. Hrsg. von Kudraß Thomas. Carl Hanser Verlag GmbH & Co. KG. ISBN: 978-3-446-43508-7. DOI: 10.3139/9783446440265.fm. URL: <https://www.hanser-elibrary.com/doi/10.3139/9783446440265.fm> (besucht am 21.06.2026).
- Microsoft Corporation (Hrsg.) (31. März 2026a). *Best Practices for Background Jobs - Azure Architecture Center*. URL: <https://learn.microsoft.com/en-us/azure/architecture/best-practices/background-jobs> (besucht am 17.06.2026).
- (12. Juni 2026b). *Queue-Based Load Leveling Pattern - Azure Architecture Center*. URL: <https://learn.microsoft.com/en-us/azure/architecture/patterns/queue-based-load-leveling> (besucht am 17.06.2026).
- Newman, Sam (Sep. 2026). *Building Resilient Distributed Systems*. O'Reilly. URL: <https://learning.oreilly.com/library/view/building-resilient-distributed/9781098163532/> (besucht am 24.05.2026).
- Richardson, Chris (2018). *Microservices patterns*. Software development. Shelter Island: Manning.
- Rosoco BV (Hrsg.) (24. Mai 2026). *JobRunr Documentation*. URL: <https://www.jobrunr.io/en/documentation/> (besucht am 24.05.2026).
- Saake, Gunter, Andreas Heuer und Kai-Uwe Sattler (Juni 2018). *Datenbanken – Konzepte und Sprachen*. mitp Verlags GmbH & Co KG. ISBN: 978-3-95845-778-2. URL: <https://learning.oreilly.com/library/view/datenbanken-konzepte/9783958457782/> (besucht am 19.06.2026).
- Temporal Technologies Inc. (24. Mai 2026). *Temporal Docs*. URL: <https://docs.temporal.io/> (besucht am 24.05.2026).
- Terracotta Inc. (Hrsg.) (24. Mai 2026). *Quartz Scheduler Documentation*. URL: <https://www.quartz-scheduler.org/documentation/quartz-2.5.x/> (besucht am 24.05.2026).

The PostgreSQL Global Development Group (Hrsg.) (14. Mai 2026). *PostgreSQL 18.4 Documentation*. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/18/index.html> (besucht am 21.06.2026).